

結合 Google Apps Script 串接 Google Sheets 管理後台的動態資訊網頁



一、建立 Google 試算表

先在個人的 Google 雲端硬碟建立一個全新的「Google 試算表」，並將工作表命名為 members。設定以下欄位（第一列為標題列）：

A (欄)	B (欄)	C (欄)	D (欄)
id	name	email	role
1	張小明	xiaoming@example.com	管理員
2	李小華	xiaohua@example.com	一般會員

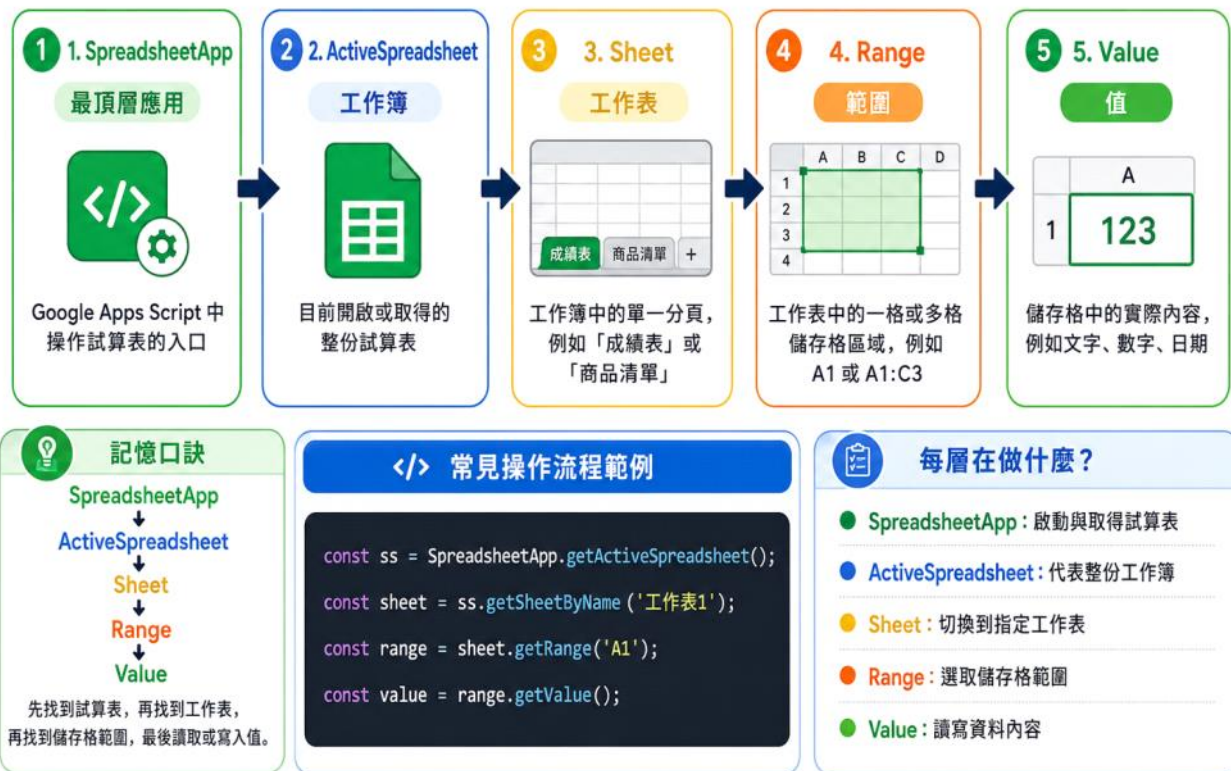
二、Google Apps Script 環境設定與基礎操作

1. 什麼是 Google Apps Script (GAS)

Google Apps Script 是一款基於 JavaScript (ES6+) 的雲端指令碼平台。它直接運行於 Google 伺服器上，免去本地端環境部署（如 Node.js、伺服器租借等）的麻煩。在開始寫程式之前，必須理解 Google 試算表 API 的層級關係：

Google 試算表 API 的層級關係

從最上層應用到底層儲存格值，逐層操作 Google Sheets 資料



相關函式：

- SpreadsheetApp：GAS 中操作試算表的入口物件。
- getActiveSpreadsheet()：取得目前這個與指令碼繫結的活頁簿。
- getSheetByName("工作表名稱")：精準取得特定名稱的分頁。
- getDataRange()：自動選取該工作表中有資料的最小矩形範圍。
- getValues()：取得範圍內的數值，回傳格式為 **二維陣列**（例如：[[row1_col1, row1_col2], [row2_col1, row2_col2]]）。

2. Prompt 生成 GAS 基礎讀寫程式碼：

你現在是一位專業的 Google Apps Script (GAS) 開發專家。請幫我寫一份 GAS 程式碼。

我的需求如下：

1. 請寫一個名為 readSheetData 的函式。

- 讀取目前繫結試算表中，名為 'members' 的工作表。
- 使用 getDataRange() 取得所有資料。
- 將讀取到的二維陣列，轉換為『以第一列為標題 (Key)』的 JSON 物件陣列（排除標題列）。
- 最後使用 Logger.log() 印出格式化後的 JSON 字串。

2. 請寫一個名為 writeSheetData 的函式。

- 讀取 'members' 工作表。
- 自動計算目前的最後一列行數作為新的 ID。

· 使用 `appendRow` 寫入一筆模擬資料 (欄位順序 : `id`, `name`, `email`, `role`) 。

AI 生成的參考程式碼

```
/**
 * 讀取 members 工作表並轉為 JSON 物件陣列
 */
function readSheetData() {
  const spreadsheet = SpreadsheetApp.getActiveSpreadsheet();
  const sheet = spreadsheet.getSheetByName("members");

  if (!sheet) {
    Logger.log("錯誤：找不到名為 members 的工作表！");
    return;
  }

  // 取得所有有資料的範圍與數值 (二維陣列)
  const range = sheet.getDataRange();
  const values = range.getValues();

  const headers = values[0]; // 取得第一列標題
  const formattedData = [];

  // 從第二列 (索引 1) 開始遍歷資料
  for (let i = 1; i < values.length; i++) {
    const row = values[i];
    const item = {};
    headers.forEach((header, index) => {
      item[header] = row[index];
    });
    formattedData.push(item);
  }

  // 輸出至執行日誌
  Logger.log("格式化後的資料：\n" + JSON.stringify(formattedData, null, 2));
}

/**
 * 自動產生遞增 ID 並寫入一筆新會員資料
 */
function writeSheetData() {
  const spreadsheet = SpreadsheetApp.getActiveSpreadsheet();
  const sheet = spreadsheet.getSheetByName("members");

  if (!sheet) {
```

```

    Logger.log("錯誤：找不到工作表！");
    return;
  }

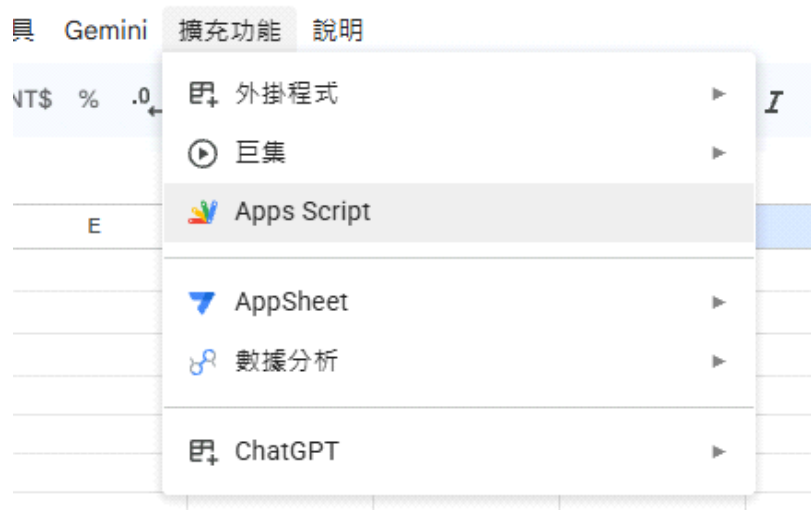
  const newId = sheet.getLastRow(); // 自動以目前最後一列行數作為新 ID
  const newName = "陳大同";
  const newEmail = "datong@example.com";
  const newRole = "一般會員";

  // 附加資料至工作表最後一行
  sheet.appendRow([newId, newName, newEmail, newRole]);
  Logger.log("成功寫入資料：ID=" + newId);
}

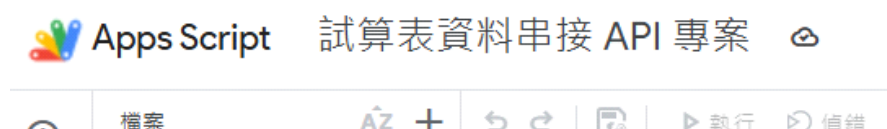
```

3. GAS操作流程與權限授權與偵錯 (Debug)

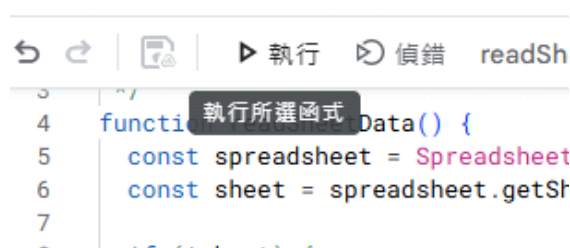
- (1) 點選 Google 試算表上方選單「擴充功能」->「Apps Script」



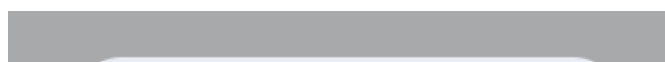
- (2) 變更專案名稱

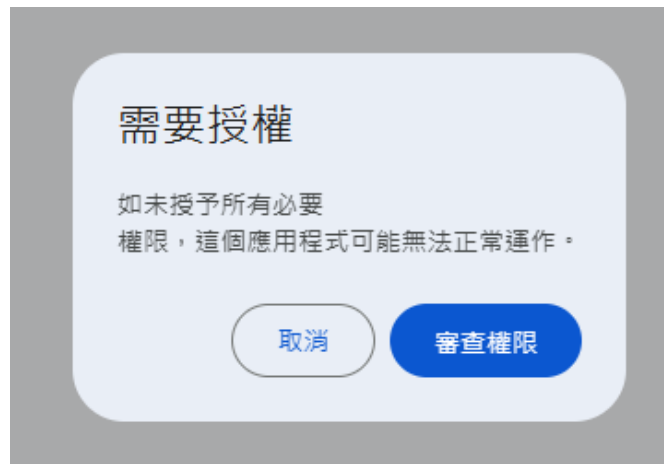


- (3) 貼上程式碼，儲存專案後點選執行



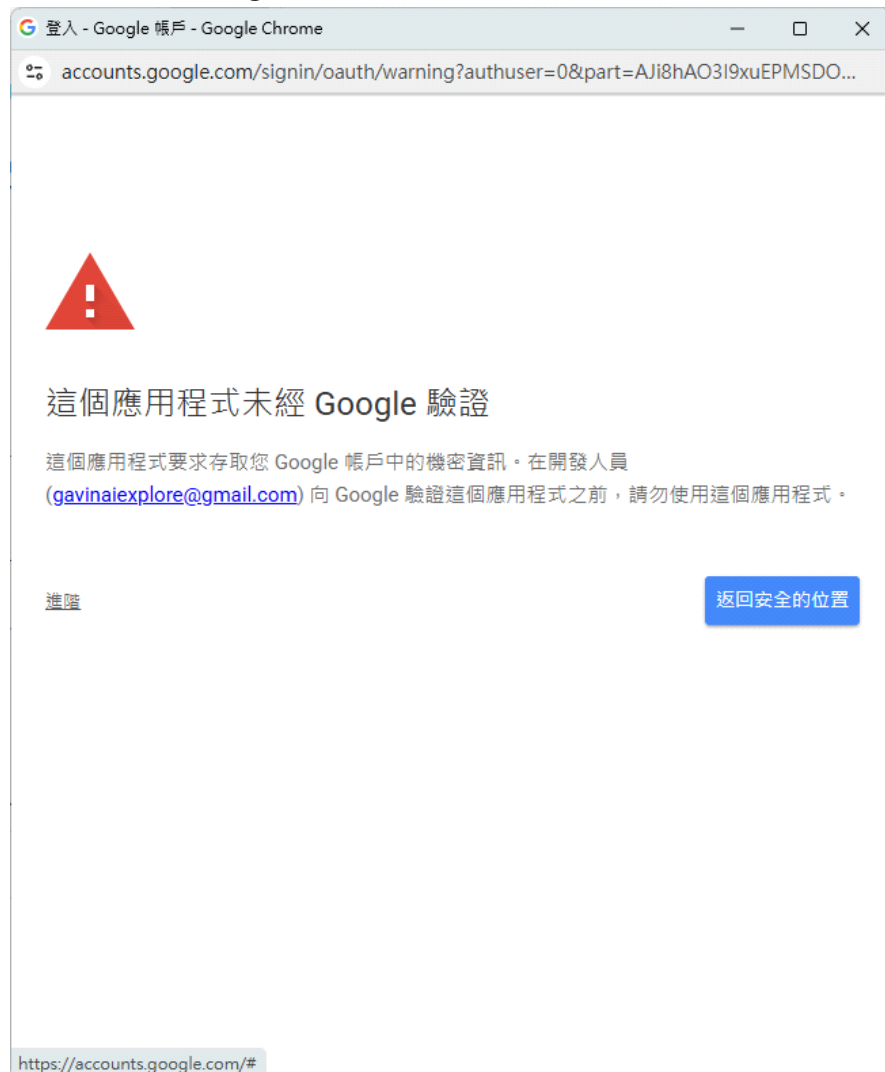
- (4) 系統彈出「需要授權」提示，請點選「審查權限」





(5) 選擇目前的 Google 帳戶 (若有)

(6) 當畫面顯示「Google 尚未驗證此應用程式」時，請點選左下角的「進階」



(7) 點選「前往『未命名專案』(不安全)」

隱藏進階設定

除非您瞭解相關風險並信任開發人員 ([gavinaie](#)

[前往「試算表資料串接 API 專案」\(不安全\)](#)

(8) 確認要求後，點選「繼續」

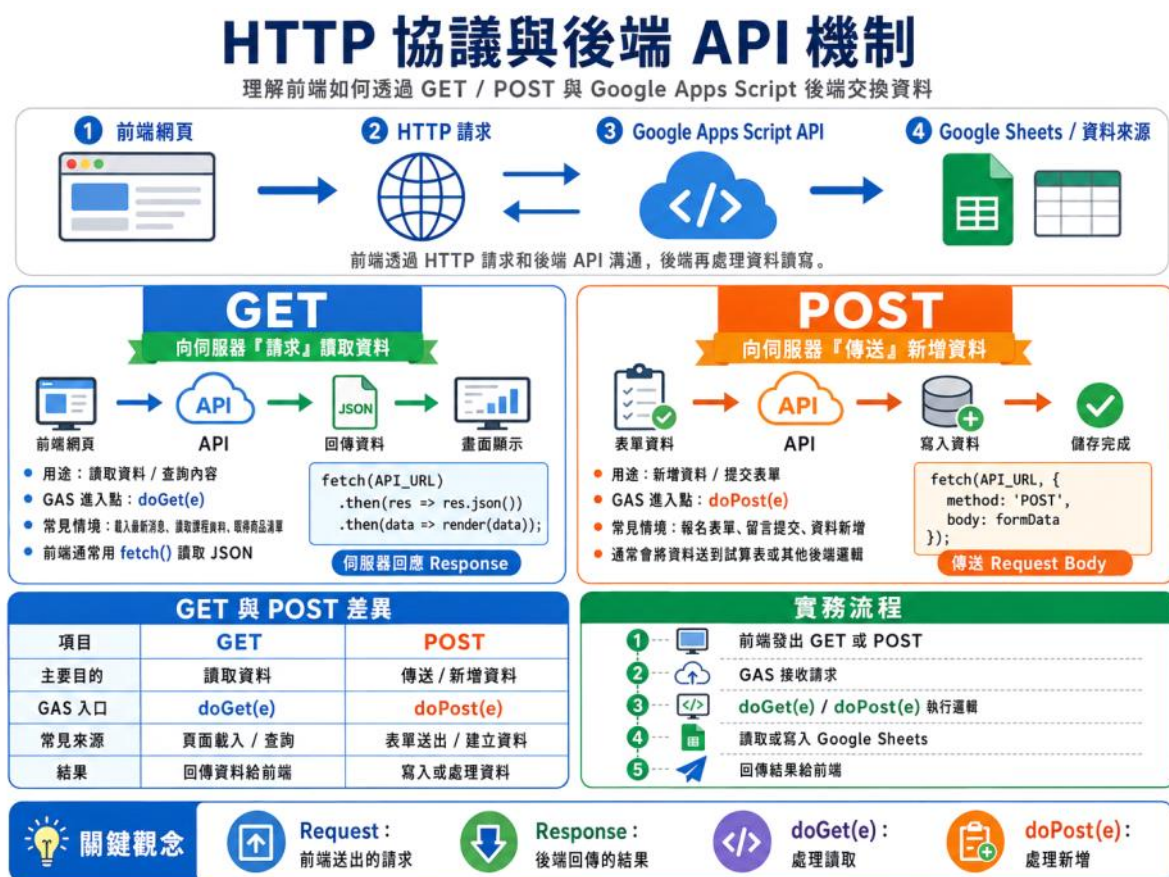


(9) 授權完成後即可在下方的「執行日誌」中看到印出的資料

執行記錄		
下午1:38:28	通知	開始執行
下午1:38:29	資訊	格式化後的資料： <pre>[{ "id": 1, "name": "張小明", "email": "xiaoming@example.com", "role": "管理員" }, { "id": 2, "name": "李小華", "email": "xiaohua@example.com", "role": "一般會員" }]</pre>
下午1:38:29	通知	執行完畢

二、將試算表轉換為 Web API

1. HTTP 協定與後端 API 機制



(1) 將 GAS 部署為網頁應用程式 (Web App) 時，它就具備了伺服器的功能。外部用戶端可透過 HTTP 方法與之通訊：

- 1) **GET**：向伺服器「請求」讀取資料。GAS 的進入點為 `doGet(e)`。
- 2) **POST**：向伺服器「傳送」新增資料。GAS 的進入點為 `doPost(e)`。

(2) 這兩個函式名稱是 Google 系統保留字，大小寫必須完全一致（例如不可以寫成 `doget` 或 `DOPOST`），且兩者都接受一個代表「事件物件」的參數 `e`，其中包含請求所攜帶的所有參

數與內容。

2. Prompt 生成 API 路由與 JSON 回傳控制程式碼：

(1) 你現在是一位後端 API 開發專家。請幫我寫一個完整的 Google Apps Script Web API 程式碼。需求如下：

1.請實作 doGet(e) 函式：

- 接收一個查詢參數 action。
- 若 action === "read"，讀取試算表中 'members' 工作表的所有資料（不含標題列），將其轉為 JSON 物件陣列。
- 回傳一個標準的 JSON 格式回應，結構為：{ status: "success", data: [...] }。
- 若 action 參數不正確，回傳錯誤 JSON：{ status: "error", message: "不支援的 action" }。

2.請實作 doPost(e) 函式：

- 接收外部傳入的 JSON Body 資料（包含 name, email, role）。
- 解析該 JSON 內容（請注意，前端為了避免 CORS 跨來源資源共用限制，可能會以 text/plain 格式傳入，請使用 JSON.parse(e.postData.contents) 進行安全解析）。
- 將解析後的資料寫入 'members' 工作表（ID 需自動遞增）。
- 寫入成功後，回傳：{ status: "success", message: "資料新增成功", data: { 剛寫入的物件 } }。

3.請封裝兩個輔助函式：responseSuccess(data, message) 與 responseError(message)，統一使用 ContentService.createTextOutput 來輸出 MimeType.JSON 格式，避免重寫重複的程式碼。

4.請加入完整的 try...catch 錯誤處理。

(2) AI 生成的參考程式碼

```
/**
 * 處理外部 GET 請求
 * 範例網址：https://.../exec?action=read
 */
function doGet(e) {
  try {
    const action = e.parameter.action;

    if (action === "read") {
      const spreadsheet = SpreadsheetApp.getActiveSpreadsheet();
      const sheet = spreadsheet.getSheetByName("members");
      const values = sheet.getDataRange().getValues();
```

```

const headers = values[0];
const data = [];

// 轉換為 JSON 物件陣列
for (let i = 1; i < values.length; i++) {
  const row = values[i];
  const item = {};
  headers.forEach((header, index) => {
    item[header] = row[index];
  });
  data.push(item);
}
return responseSuccess(data, "資料讀取成功");
} else {
  return responseError("無效的 action 參數");
}
} catch (error) {
  return responseError("伺服器發生 GET 錯誤：" + error.toString());
}
}
}

```

```

/**
 * 處理外部 POST 請求
 */
function doPost(e) {
  try {
    // 解析外部傳來的 JSON 字串 (即使 Content-Type 為 text/plain 也能安全解析)
    const postData = JSON.parse(e.postData.contents);
    const name = postData.name;
    const email = postData.email;
    const role = postData.role || "一般會員";

    if (!name || !email) {
      return responseError("缺少必要欄位：name 或 email");
    }

    const spreadsheet = SpreadsheetApp.getActiveSpreadsheet();
    const sheet = spreadsheet.getSheetByName("members");
    const newId = sheet.getLastRow(); // 自動計算 ID

```

```

// 寫入試算表
sheet.appendRow([newId, name, email, role]);

const insertedData = { id: newId, name: name, email: email, role: role };
return responseSuccess(insertedData, "資料新增成功");
} catch (error) {
return responseError("伺服器發生 POST 錯誤 : " + error.toString());
}
}

/* --- 統一 JSON 回傳格式之輔助函式 --- */

function responseSuccess(data, message) {
const result = {
status: "success",
message: message,
data: data
};
return ContentService.createTextOutput(JSON.stringify(result))
.setMimeType(ContentService.MimeType.JSON);
}

function responseError(message) {
const result = {
status: "error",
message: message
};
return ContentService.createTextOutput(JSON.stringify(result))
.setMimeType(ContentService.MimeType.JSON);
}

```

(3) 清空 Apps Script 中的程式碼，再貼上上面生成的的程式碼，儲存專案後點選執行

3. Web App 部署與版本控制管理機制

(1) 點選「部署」->「新增部署」



(2) 點選左上角設定圖示，選取「網頁應用程式」



(3) 填入說明後，選取以下設定：

- 1) 執行身分：必須選取「(您的 Google 帳戶)」，這代表外部請求會借用您的管理員身份去寫入這張 Sheets
- 2) 誰可以存取：必須選取「所有人 (Anyone)」，否則前端串接會遇到 401 權限阻擋



(4) 點選部署並複製生成的 Web App 網址



- (5) 複製網頁應用程式網址，在網址最後面加上 ?action=read 再按 Enter 測試，就能順利看到試算表的 JSON 資料

如：<https://script.google.com/macros/s/ID/exec?action=read>

```
{
  "status": "success",
  "message": "資料讀取成功",
  "data": [
    {
      "id": 1,
      "name": "張小明",
      "email": "xiaoming@example.com",
      "role": "管理員"
    },
    {
      "id": 2,
      "name": "李小華",
      "email": "xiaohua@example.com",
      "role": "一般會員"
    }
  ]
}
```

※版本控制規則（重要）：

GAS 的 Web App 網址在每次程式碼變更後，不能直接點選「新增部署」，這樣會產生一個新的 URL，造成前端需重複修改網址。正確更新作法：點選「部署」->「管理部署」->點選「編輯(鉛筆圖示)」->版本選擇「新版本」->按下部署，即可在「不改變 API URL」的情況下更新後端程式碼。。

三、Fetch API 非同步請求原理與實戰

Fetch API 非同步請求原理

理解 JavaScript 非同步流程、Promise 演進，以及前端 CORS 與 Redirect 的常見陷阱



1. JavaScript 的非同步 (Asynchronous) 運作原理

JavaScript 採用「單執行緒 (Single Thread)」架構，代表它同一時間只能執行一個程式碼區塊。如果發送網路請求，並採用「同步 (Synchronous)」寫法，瀏覽器在等待伺服器回應 (可能需要 1~2 秒) 的期間會進入完全卡死、無法點擊的狀態。為了避免阻塞，需使用非同步機制：

- (1) **Callback 回呼函式**：最傳統的做法，但過多的巢狀縮排會導致「回呼地獄 (Callback Hell)」。
- (2) **Promise 物件**：解決回呼地獄，提供 .then() 與 .catch() 的鏈式呼叫。
- (3) **Async / Await (ES8+)**：最現代的語法糖，讓非同步程式碼看起來與同步程式碼一樣乾淨直覺。

2. 前端 CORS 與重導向 (Redirect) 的底層陷阱

當瀏覽器發送跨網域請求時，如果設定 headers: { 'Content-Type': 'application/json' }，瀏覽器會依據安全規範，先發送一個 OPTIONS (預檢請求) 到 GAS 伺服器，但 Google 的 Web App 機制並不支援 OPTIONS 請求，因此會直接回傳錯誤。這就是最令人頭痛的 CORS (Cross-Origin Resource Sharing) 跨來源資源共用錯誤。

※AI 協作避開 CORS 技巧：

在發送 POST 時，將 Content-Type 設定為 text/plain (純文字)，但本體 (Body) 依然傳送 JSON 字串。GAS 端會把這段純文字當作 contents 接收，再在 GAS 內用 JSON.parse() 解析。這樣既能傳遞 JSON，又不會觸發瀏覽器的 CORS 阻擋！

四、AI 協作實戰 - 一鍵生成 Antigravity 懸浮科技風資料串接網頁

1. Prompt 生成 Antigravity 反重力視覺網頁與非同步串接程式碼：

你現在是一位精通前端開發、資料串接，且瘋狂熱愛前衛『Antigravity (反重力/輕量懸浮)』視覺風格的頂級網頁工程師。請幫我寫一個單一檔案的 index.html 網頁。需求如下：

1. Antigravity (反重力) 視覺風格設計：

- 主色調：採用深邃幽靜的宇宙星空背景（例如：深色底 bg-slate-950），搭配具有未來感的神祕色彩漸層。
- 無重力懸浮感：在 CSS 中自訂 @keyframes float 動態效果，讓畫面上的表單卡片與表格卡片產生輕微、緩慢地上下飄浮（Floating）視覺特效，使其具有完美的「重力消失」感。
- 毛玻璃效果 (Glassmorphism)：所有的資訊卡片與容器必須具有半透明霧面質感（backdrop-blur-md bg-white/5），並搭配精緻的白邊微光細線（border border-white/10）。
- 霓虹發光與 Hover 回饋：所有輸入框與按鈕在 Focus 或 Hover 時，需有向外擴散的亮綠色或螢光藍霓虹陰影（Neon Glow），並且卡片微微向上位移，提供完美的漂浮回饋感。

2. 資料讀取 (GET 請求)：

- 宣告一個全域變數 API_URL（請在程式碼中留空或使用占位符提醒我更換）。
- 寫一個非同步函式 fetchMembers()，使用 fetch() 對 \${API_URL}?action=read 發送 GET 請求。
- 當正在讀取資料時，在表格區域顯示一個霓虹流光旋轉 Loading 動畫。
- 讀取成功後，動態將資料渲染至表格中；並根據不同的角色等級（如管理員、進階會員、一般會員）給予不同霓虹發光色彩的懸浮 Badge（標籤）樣式。

3. 資料寫入 (POST 請求)：

- 監聽表單的提交事件 (submit)，阻止預設行為。
- 將按鈕轉為 Loading 載入狀態，防止重複點擊。
- 使用 fetch() 將表單資料以 POST 方法傳送至 API_URL。
- 注意：為了避開 Google Apps Script 的 CORS 限制，必須將 headers 設為 { 'Content-Type': 'text/plain' }，並將資料以 JSON.stringify(payload) 方式傳入。
- 寫入成功後，在畫面上顯示一個精美的美化提示框（非瀏覽器預設 alert），清空表單，並自動重新呼叫 fetchMembers() 刷新右側表格。

2. AI 生成的參考程式碼

```
<!DOCTYPE html>
<html lang="zh-TW">
<head>
  <meta charset="UTF-8">
```

```

<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Google Sheets API - Antigravity 懸浮串接端</title>
<!-- 引入 Tailwind CSS 確保極佳的 RWD 外觀 -->
<script src="[https://cdn.tailwindcss.com](https://cdn.tailwindcss.com)"></script>
<!-- 引入 Lucide 圖標 -->
<script src="[https://unpkg.com/lucide@latest](https://unpkg.com/lucide@latest)">
</script>
<style>
/* 自訂反重力緩慢懸浮動畫 (Antigravity Floating Effect) */
@keyframes float-y {
  0%, 100% {
    transform: translateY(0px);
    box-shadow: 0 25px 50px -12px rgba(16, 185, 129, 0.1);
  }
  50% {
    transform: translateY(-8px);
    box-shadow: 0 25px 50px -4px rgba(16, 185, 129, 0.2);
  }
}

@keyframes float-y-delayed {
  0%, 100% {
    transform: translateY(0px);
    box-shadow: 0 25px 50px -12px rgba(59, 130, 246, 0.1);
  }
  50% {
    transform: translateY(-8px);
    box-shadow: 0 25px 50px -4px rgba(59, 130, 246, 0.2);
  }
}

.antigravity-float-1 {
  animation: float-y 6s ease-in-out infinite;
}

.antigravity-float-2 {
  animation: float-y-delayed 6s ease-in-out infinite;
  animation-delay: 2s;
}
</style>
</head>
<body class="bg-slate-950 min-h-screen text-slate-100 flex items-center justify-center
relative overflow-x-hidden">
<!-- 背景流光點綴 · 增強宇宙空間深邃感 -->
<div class="absolute top-1/4 left-1/4 w-96 h-96 bg-emerald-500/10 rounded-full filter blur-
[120px] pointer-events-none"></div>

```

```

<div class="absolute bottom-1/4 right-1/4 w-96 h-96 bg-blue-500/10 rounded-full filter
blur-[120px] pointer-events-none"> </div>
<div class="max-w-6xl w-full mx-auto px-6 py-12 relative z-10">
  <!-- 頁首標題 -->
  <header class="mb-12 text-center lg:text-left">
    <h1 class="text-4xl font-extrabold text-white tracking-tight flex items-center justify-
center lg:justify-start gap-3">
      <i data-lucide="orbit" class="text-emerald-400 w-10 h-10 animate-spin"
style="animation-duration: 20s;"> </i>
      Google Sheets <span class="bg-gradient-to-r from-emerald-400 to-cyan-400 bg-clip-
text text-transparent">Antigravity</span> 串接端
    </h1>
    <p class="text-slate-400 mt-3 text-lg font-light">基於無重力輕量懸浮美學 · 零延遲同步
Google 雲端試算表資料</p>
  </header>
  <div class="grid grid-cols-1 lg:grid-cols-3 gap-8 items-start">
    <!-- 左側欄：反重力懸浮表單 -->
    <div class="lg:col-span-1 bg-white/5 backdrop-blur-xl p-8 rounded-3xl border border-
white/10 antigravity-float-1 shadow-2xl transition-all duration-300 hover:border-
emerald-500/30">
      <h2 class="text-2xl font-bold text-white mb-6 flex items-center gap-3">
        <i data-lucide="user-plus" class="text-emerald-400"> </i>
        新增新成員
      </h2>
      <form id="memberForm" class="space-y-5">
        <div>
          <label class="block text-xs uppercase tracking-wider font-semibold text-slate-400
mb-2">姓名</label>
          <input type="text" id="nameInput" required placeholder="例如：張小明"
class="w-full px-4 py-3 bg-white/5 border border-white/10 rounded-2xl text-white
placeholder-slate-500 focus:outline-none focus:ring-2 focus:ring-emerald-500 focus:bg-
white/10 focus:border-transparent transition-all duration-300 hover:border-white/20">
        </div>
        <div>
          <label class="block text-xs uppercase tracking-wider font-semibold text-slate-400
mb-2">電子信箱</label>
          <input type="email" id="emailInput" required placeholder="xiaoming@example.com"
class="w-full px-4 py-3 bg-white/5 border border-white/10 rounded-2xl text-white
placeholder-slate-500 focus:outline-none focus:ring-2 focus:ring-emerald-500 focus:bg-
white/10 focus:border-transparent transition-all duration-300 hover:border-white/20">
        </div>
        <div>
          <label class="block text-xs uppercase tracking-wider font-semibold text-slate-400

```

```

mb-2">角色權限</label>
    <select id="roleInput"
      class="w-full px-4 py-3 bg-slate-900 border border-white/10 rounded-2xl text-
white focus:outline-none focus:ring-2 focus:ring-emerald-500 transition-all duration-300
hover:border-white/20">
      <option value="一般會員">一般會員</option>
      <option value="進階會員">進階會員</option>
      <option value="管理員">管理員</option>
    </select>
  </div>
  <button type="submit" id="submitBtn"
    class="w-full bg-gradient-to-r from-emerald-500 to-teal-500 hover:from-
emerald-400 hover:to-teal-400 text-slate-950 font-bold py-3.5 rounded-2xl shadow-lg
shadow-emerald-500/20 hover:shadow-emerald-500/40 hover:-translate-y-0.5 transition-all
duration-300 flex items-center justify-center gap-2">
    <span id="submitText">寫入試算表</span>
    <div id="submitSpinner" class="hidden w-5 h-5 border-2 border-slate-950 border-t-
transparent rounded-full animate-spin"></div>
  </button>
</form>

<!-- 美化提示訊息框 -->
<div id="alertBox" class="hidden mt-5 p-4 rounded-2xl text-sm border transition-all
duration-500"></div>
</div>

<!-- 右側欄：反重力懸浮表格 -->
<div class="lg:col-span-2 bg-white/5 backdrop-blur-xl p-8 rounded-3xl border border-
white/10 antigravity-float-2 shadow-2xl transition-all duration-300 hover:border-blue-500/30
flex flex-col min-h-[460px]">
  <div class="flex justify-between items-center mb-6">
    <h2 class="text-2xl font-bold text-white flex items-center gap-3">
      <i data-lucide="users" class="text-cyan-400"></i>
      雲端成員狀態
    </h2>
    <!-- 重新整理按鈕 -->
    <button id="refreshBtn" class="flex items-center gap-1.5 px-4 py-2 text-sm font-
semibold text-slate-300 hover:text-white bg-white/5 hover:bg-white/10 border border-
white/10 rounded-xl transition-all duration-300">
      <i data-lucide="refresh-cw" class="w-4 h-4"></i>
      重新整理
    </button>
  </div>

  <!-- 骨架載入動畫 (Loading Spinner) -->
  <div id="loadingState" class="space-y-4 my-auto">

```

```

<div class="flex justify-center items-center flex-col gap-4 py-16">
  <div class="relative w-12 h-12">
    <div class="absolute inset-0 border-4 border-emerald-500/20 rounded-full"> </div>
    <div class="absolute inset-0 border-4 border-emerald-400 border-t-transparent
rounded-full animate-spin"> </div>
  </div>
  <p class="text-slate-400 font-medium animate-pulse tracking-widest text-sm">正在突
破重力，向雲端請求資料...</p>
</div>
</div>
<!-- 空資料狀態 -->
<div id="emptyState" class="hidden my-auto py-16 text-center">
  <i data-lucide="package-open" class="mx-auto w-14 h-14 text-slate-600 mb-3"> </i>
  <p class="text-slate-500 font-medium">試算表內尚未存放任何資料</p>
</div>
<!-- 資料表格 -->
<div id="tableContainer" class="hidden overflow-x-auto rounded-2xl border border-
white/10 bg-slate-950/40">
  <table class="w-full text-left border-collapse">
    <thead>
      <tr class="bg-white/5 border-b border-white/10">
        <th class="py-4 px-6 text-xs font-bold uppercase tracking-widest text-slate-400">
ID</th>
        <th class="py-4 px-6 text-xs font-bold uppercase tracking-widest text-slate-400">姓
名</th>
        <th class="py-4 px-6 text-xs font-bold uppercase tracking-widest text-slate-400">電
子信箱</th>
        <th class="py-4 px-6 text-xs font-bold uppercase tracking-widest text-slate-400">身
份層級</th>
      </tr>
    </thead>
    <tbody id="memberTableBody" class="divide-y divide-white/5">
      <!-- 資料由 JS 動態生成 -->
    </tbody>
  </table>
</div>
</div>
</div>
</div>
<script>
  // □ 貼心提示：請學員將下方網址替換為自己從 Apps Script 部署得到的 Web App 網址！
  const API_URL = "[https://script.google.com/macros/s/這裡請換成您的GAS部署網址/exec]
(https://script.google.com/macros/s/這裡請換成您的GAS部署網址/exec)";
  // DOM 元素選取

```

```

const memberForm = document.getElementById("memberForm");
const nameInput = document.getElementById("nameInput");
const emailInput = document.getElementById("emailInput");
const roleInput = document.getElementById("roleInput");
const submitBtn = document.getElementById("submitBtn");
const submitText = document.getElementById("submitText");
const submitSpinner = document.getElementById("submitSpinner");
const alertBox = document.getElementById("alertBox");
const refreshBtn = document.getElementById("refreshBtn");

const loadingState = document.getElementById("loadingState");
const emptyState = document.getElementById("emptyState");
const tableContainer = document.getElementById("tableContainer");
const memberTableBody = document.getElementById("memberTableBody");
// 初始化 Lucide 圖標
lucide.createIcons();
/**
 * 從 API 讀取資料並渲染表格 (GET 請求)
 */
async function fetchMembers() {
  loadingState.classList.remove("hidden");
  tableContainer.classList.add("hidden");
  emptyState.classList.add("hidden");
  try {
    const response = await fetch(`${API_URL}?action=read`);
    const result = await response.json();
    if (result.status === "success" && result.data.length > 0) {
      renderTable(result.data);
      loadingState.classList.add("hidden");
      tableContainer.classList.remove("hidden");
    } else {
      loadingState.classList.add("hidden");
      emptyState.classList.remove("hidden");
    }
  } catch (error) {
    console.error("讀取失敗:", error);
    loadingState.classList.add("hidden");
    showAlert("對雲端硬碟讀取失敗，請確認 API 端點網址正確。", "error");
  }
}
/**
 * 渲染資料至 HTML Table Body
 */
function renderTable(members) {

```

```

memberTableBody.innerHTML = "";
members.forEach(member => {
  const rowHtml = `
    <tr class="hover:bg-white/5 transition-all duration-200">
      <td class="py-4 px-6 font-mono text-sm text-slate-400">${member.id}</td>
      <td class="py-4 px-6 font-semibold text-white">${escapeHTML(member.name)}</td>
      <td class="py-4 px-6 text-slate-300">${escapeHTML(member.email)}</td>
      <td class="py-4 px-6">
        <span class="inline-flex px-3 py-1 text-xs font-bold rounded-full
${getRoleBadgeClass(member.role)}">
          ${escapeHTML(member.role)}
        </span>
      </td>
    </tr>
  `;
  memberTableBody.insertAdjacentHTML("beforeend", rowHtml);
});
}
/**
 * 新增會員資料 (POST 請求)
 */
memberForm.addEventListener("submit", async (e) => {
  e.preventDefault();
  setSubmitLoading(true);
  hideAlert();
  const payload = {
    name: nameInput.value,
    email: emailInput.value,
    role: roleInput.value
  };
  try {
    // 以 text/plain 發送 JSON 字串，完美避開 CORS 阻擋
    const response = await fetch(API_URL, {
      method: "POST",
      headers: {
        "Content-Type": "text/plain"
      },
      body: JSON.stringify(payload)
    });
    const result = await response.json();
    if (result.status === "success") {
      showAlert("恭喜！資料成功穿越重力屏障，寫入試算表！", "success");
      memberForm.reset();
      fetchMembers(); // 動態更新右側資料
    }
  }
});

```

```

    } else {
        showAlert(`新增失敗: ${result.message}`, "error");
    }
} catch (error) {
    console.error("提交失敗:", error);
    showAlert("寫入失敗 · 請確認 Apps Script 已成功授權且部署為全新版本。", "error");
} finally {
    setSubmitLoading(false);
}
});
// 重新整理按鈕事件
refreshBtn.addEventListener("click", fetchMembers);
/* --- 實用小工具函式 (Utility Functions) --- */
function getRoleBadgeClass(role) {
    switch (role) {
        case "管理員":
            return "bg-rose-500/15 text-rose-300 border border-rose-500/30 shadow-[0_0_10px_rgba(244,63,94,0.1)]";
        case "進階會員":
            return "bg-cyan-500/15 text-cyan-300 border border-cyan-500/30 shadow-[0_0_10px_rgba(6,182,212,0.1)]";
        default:
            return "bg-slate-500/15 text-slate-300 border border-slate-500/30";
    }
}
function setSubmitLoading(isLoading) {
    if (isLoading) {
        submitBtn.disabled = true;
        submitText.textContent = "發送資料中...";
        submitSpinner.classList.remove("hidden");
        submitBtn.classList.add("opacity-70", "cursor-not-allowed");
    } else {
        submitBtn.disabled = false;
        submitText.textContent = "寫入試算表";
        submitSpinner.classList.add("hidden");
        submitBtn.classList.remove("opacity-70", "cursor-not-allowed");
    }
}
function showAlert(message, type) {
    alertBox.classList.remove("hidden", "bg-emerald-500/10", "text-emerald-400", "border-emerald-500/20", "bg-rose-500/10", "text-rose-400", "border-rose-500/20");
    if (type === "success") {
        alertBox.classList.add("bg-emerald-500/10", "text-emerald-400", "border", "border-emerald-500/20");
    }
}

```

```

    } else {
        alertBox.classList.add("bg-rose-500/10", "text-rose-400", "border", "border-
rose-500/20");
    }
    alertBox.textContent = message;
}
function hideAlert() {
    alertBox.classList.add("hidden");
}
function escapeHTML(str) {
    return str.replace(/[\&<>"]/g,
        tag => ({ '&': '&amp;', '<': '&lt;', '>': '&gt;', '": '&#39;', '": '&quot;' }[tag] || tag)
    );
}
// 頁面初次載入時自動拉取資料
window.addEventListener("DOMContentLoaded", () => {
    if (API_URL.includes("這裡請換成您的GAS部署網址")) {
        showAlert("請先修改 index.html 中第 121 行的 API_URL · 替換為您個人的 GAS Web App 部
署網址！", "error");
        loadingState.classList.add("hidden");
        emptyState.classList.remove("hidden");
    } else {
        fetchMembers();
    }
});
</script>
</body>
</html>

```

3. 網頁畫面



五、常見錯誤排查與偵錯 (Debug)

1. 常見錯誤

常見錯誤排查與偵錯 (Debug)

Google Apps Script 串接 Google Sheets 與前端 Fetch API 時，最常見的 3 種錯誤與手動修復方式

1 > 觀察 Console 錯誤
2 🔍 判斷原因
3 API 手動驗證
4 🛠 修正並重新部署

錯誤一：Failed to fetch / CORS policy

症狀

Console 出現 Failed to fetch 並伴隨 CORS policy 警告

原因分析

- 後端代碼.gs 的部署設定中，『誰有權存取 (Who has access)』沒有設定為『所有人 (Anyone)』，導致 Google 伺服器拒絕跨網域連線。
- 前端 fetch 時的 Content-Type 被設定為 application/json，觸發了 OPTIONS 預檢請求。

手動修復

- 重新進入 Apps Script『管理部署』，確認權限已開至『所有人』。
- 檢查前端的 headers，確保 Content-Type 為 text/plain。

小提醒

關鍵字：CORS、OPTIONS、Who has access、Anyone

錯誤二：Unexpected token < in JSON

症狀

API 回傳 status: 'error' message 顯示：SyntaxError: Unexpected token < in JSON...

原因分析

前端使用 response.json() 嘗試解析時，GAS 其實丟出了 HTML 格式的錯誤網頁（通常是權限未開或指令碼損毀）。

手動修復

- 直接將 Web App 的 URL 貼在瀏覽器網址列，手動加上 ?action=read 進行測試。
- 觀察是否輸出乾淨的 JSON 內容，或出現 Google 系統的紅色異常警報。

小提醒

看到 < 通常代表回來的不是 HTML，不是真正的 JSON

錯誤三：讀到的資料仍是舊版本

症狀

試算表寫入成功但讀取的資料依然是舊版本

原因分析

GAS 的 Web App 部署未正確更新新版本。

手動修復

- 切記每次修改代碼.gs 後，必須使用『管理部署』更新為『新版本』部署。
- 直接點選『執行』或直接『儲存』，無法將新程式碼更新到 Web 網址上的！

小提醒

程式改了 ≠ Web App 已更新，必須重新部署版本

快速排查對照表

錯誤類型	優先檢查	常見修正方向
1 Failed to fetch / CORS	檢查 Anyone 權限	Content-Type 改 text/plain
2 Unexpected token <	瀏覽器直接測試 URL ?action=read	確認回傳 JSON，不是 HTML
3 資料是舊的	檢查是否重新部署新版本	不要只按儲存或執行

Debug 關鍵觀念

- Console: 先看錯誤訊息
- CORS: 跨來源權限
- JSON: 前後端資料格式
- 部署: 版本是否更新
- 手動驗證: 先用瀏覽器直接測試 API

2. Prompt 生成 解決 CORS / 串接報錯：

我正在將前端的 HTML (使用 Fetch API) 串接到 Google Apps Script (GAS) 部署的 Web

Vibe Coding-V1 第 23 頁

API。目前在瀏覽器的 Console (主控台) 中遇到了以下錯誤：[請將 DevTools → Console 中中紅色錯誤代碼或文字複製貼在這邊]

以下是我的前端 fetch 程式碼：[貼上 index.html 的 fetch 區塊]

以下是我的 GAS 程式碼：[貼上 代碼.gs]

請幫我：

- 1.解釋為什麼會產生這個錯誤。
- 2.提供修復後的完整正確程式碼。

3. Prompt 生成 為試算表 API 新增功能 (如刪除、搜尋)：

我目前有一個運作良好的 Google Apps Script Web API。我的試算表欄位有：id, name, email, role。

我希望在不破壞現有 doGet 與 doPost 邏輯的情況下，新增一個功能：

- 1.請在 doPost 中新增一個判斷：如果傳入的 JSON 含有 action: "delete" 且包含 id，則去試算表中找到該 id 並刪除該整列。
- 2.請提供我前端如何撰寫對應的 fetch DELETE 請求程式碼。

請用台灣繁體中文撰寫，並提供修復後的程式碼與使用說明。